안동주 (An Dongju) Frontend Engineer

+82 10-6303-2184 · andongjoo1@naver.com
github.com/dev-frankie · linkedin.com/in/adj0715

ABOUT ME

셀렉트스타에서 LLM 평가 플랫폼 DATUMO의 프론트엔드를 리드하는 5년차 개발자 안동주입니다.

담당 기능 뿐만 아니라 인증, 권한, 결제처럼 제품 전반에 영향을 미치는 영역을 맡아, 테스트와 예외 처리, 로깅, 권한 검증까지 갖춘 상태로 일정 안에 내놓는 것을 기본으로 삼습니다. 개발 속도를 반복해서 떨어뜨리는 기술 부채와 아키텍처 경계는 고치는 데서 끝내지 않고 CI 검사로 추가하였습니다.

혼자 빨리 가는 것보다 팀이 같은 기준으로 일하게 만드는 데 관심이 많습니다. 성능과 개발 환경에서 개선할 과제를 찾아 파트원들과 함께 풀고, 결과는 수치로 증명하는 개발자입니다.

핵심 성과

파트 리딩

- 3인 파트 리드. 백엔드 6명 대비 프론트 2명이던 시기부터 개발 병목 없이 운영. 리드 기간 파트 성과로 클라이언트 번들 **–14.4%**(gzip 압축 후 2.90 → 2.37MB), 레거시 **약 17,900줄** 제거, 배포 전 자동 검사 **0 → 17개**

성능 최적화

- Sentry 실사용자 데이터에서 LCP p75 **5.3s**로 잡힌 랜딩 페이지의 원인을 하이드레이션 뒤에 실행되던 리다이렉트로 특정해 서버 307로 옮김, 랜딩 LCP **3.2 → 2.0s(–38%)**(수정 전후 빌드 Lighthouse 비교 · 5회 median)
- 평가 상세 진입은 최악 케이스 **623 → 148ms(4배)**, 중앙값 472 → 323ms, 대상 라우트 Lighthouse **99점 · CLS 0**

개발 생산성

- CI **13분 26초 → 약 4분**, 테스트 **0 → 2,100+ 케이스**, lint 오류 **56 → 0**. 검사를 통과하지 못한 코드는 merge되지 않게 연결
- 작성자(Claude) ≠ 검증자(Codex) AI 리뷰 하네스를 사내 NPM `@datumo/agent-harness` 로 배포해 **2개 레포** 적용, 릴리스 29회

아키텍처 · 인증

- 레거시 SSG를 BFF 기반 동적 구조로 전환해 **Company / Workspace / Application 3계층 권한 라우팅** 구현
- 브라우저가 토큰을 보지 못하는 세션 모델(httpOnly · 24시간 절대 상한 · 리프레시 중복 제거)
- Recoil atom **429개 → Zustand 스토어 26개**

타입 · API 계약

- OpenAPI → 타입 · Zod 자동 생성(엔드포인트 300+ · 스키마 800+)으로 수기 타입 계층을 제거하고, 스펙에 없는 경로는 컴파일 에러로
- 컴파일러 옵션 4개와 타입 정보를 쓰는 ESLint 규칙을 추가로 켜서 드러난 **221건** 전수 해소

Tech Stack TypeScript · React · Next.js · TanStack Query · Zustand · openapi-typescript · Zod · Vitest · Playwright · GitHub Actions

---

EXPERIENCE

셀렉트스타 · 프론트엔드 파트 리더

2025.07 - 현재

DATUMO Platform 개발

DATUMO Platform은 기업이 자사 LLM 애플리케이션을 평가(Eval) · 레드티밍(Red Teaming)하고 운영 중 실패 사례를 추적(Observability)하는 B2B 솔루션입니다. 고객사 온프레미스를 시작으로 글로벌 SaaS를 추진 중입니다.

저는 Criteria Setting, Run, Dashboard 등의 기능과 더불어 인증 · 권한 · BFF 같은 기반 계층을 담당했습니다.

[1] 파트 리딩 · 품질 게이트 · 개발 생산성

기술 스택 Vitest · Playwright · type-aware ESLint · GitHub Actions · Husky · gitleaks
역할 파트 리드 · 품질 체계 설계

문제

- 레거시 레포는 실행 가능한 테스트 0개, lint **56 errors / 710 warnings**, `ignoreDuringBuilds` 로 오류가 있어도 배포되는 구조라 품질이 개인 숙련도와 수동 QA에 의존
- AI 코드리뷰도 작성자와 검증자가 같은 모델이라 결과를 신뢰하기 어려운 상태

파트 리딩

- 3인 파트의 업무 배분 · 코드 리뷰 · 정기 기술 세미나 · KPT 회고 · 신규 인원 온보딩 담당, PO 요구사항 조율
- 백엔드 6명 대비 프론트 2명이던 시기부터 개발 병목 없이 운영
- 기술 부채를 테스트 · 성능 · CI/CD · 관측 · 아키텍처 · 유지보수 · 보안 7개 축으로 현행 파악해 우선순위 결정

파트 성과(과제 설정 · 배정 · 리뷰)

- 기능 개발 외 DX · 성능 · 부채 과제를 직접 발굴해 배정하고 리뷰로 마무리한 결과
- God Component 5종 **6,509 → 1,745줄(–73%)**, 구형 Red Teaming 2종 · 다크모드 등 레거시 **약 17,900줄 제거**로 클라이언트 번들 **–14.4%**, 로그인 첫 화면 배경 비디오 **–36.9%**
- knip 리포트 0(미사용 파일 77 · 의존성 8 제거)
- Sentry 이슈를 경로 · 메서드별로 분리, 소스맵 비활성화 · 난독화, GitHub Actions OIDC 전환 + 서드파티 액션 SHA 고정

merge 차단 게이트

- PR마다 lint · 타입 검사 · 테스트 · 빌드 · 번역 파일 검사를 자동 실행, 하나라도 실패하면 merge 불가
- 테스트가 코드의 몇 %를 실행하는지도 측정해 하한선(문장 82% · 분기 77% · 함수 88%) 아래로 내려가면 실패. 하한선은 도입 시점 실측값을 그대로 고정한 것으로 테스트가 늘 때마다 올리기만 함
- 측정 대상은 순수 로직이며 인증 · 모니터링처럼 부수효과가 있는 파일은 제외
- 테스트는 입력과 출력이 분명한 로직에 먼저 붙이고 새 로직은 테스트를 먼저 작성해 **0 → 2,100+ 케이스**(257 파일). QA 시나리오 문서 **321개** 정비

CI 13분 26초 → 약 4분

- 직렬 Job을 static · types · test · build · security · e2e로 병렬화하고 도구별 캐시 적용
- 커밋마다 400MB씩 쌓여 10GB 예산을 잠식하던 빌드 캐시를 날짜 단위 회전으로 정리
- 타입을 보는 lint는 캐시를 쓰면 다른 파일의 변경 영향을 놓칠 수 있어 캐시 없이 돌리고, 대신 병렬화로 시간을 회수

AI 리뷰 하네스 `@datumo/agent-harness`

- AI가 짠 코드를 같은 AI가 검토하면 같은 실수를 놓치기 때문에, 코드를 쓰는 모델(Claude)과 검토하는 모델(Codex)을 분리하고 검토자에게는 대화 기록 없이 변경 내용과 검사 결과만 전달
- 검토용 지시문은 git으로 관리해 작성자가 바꿀 수 없게 함
- 검토는 설계 단계와 코드 완성 후 두 번. 발견 사항은 심각도 3단계로 나눠 가장 높은 단계(버그 · 데이터 손실 가능)가 하나라도 있으면 merge 불가
- 통과 여부는 타입 검사 · lint · 테스트 실행 결과로만 정하고 모델의 자체 판단은 쓰지 않음
- 첫 실행에서 도구 자신의 결함 12건(타입 검사가 크래시했는데 통과로 처리하던 것 포함)을 잡아 수정
- 규칙 파일을 여러 레포에 배포할 때 각 레포의 수정을 덮어쓰지 않고 차이를 먼저 보여주는 동기화 포함. front · admin **2개 레포** 적용, 릴리스 29회

보안 거버넌스

- Dependabot 경고 **51건 전량 해소**(프로덕션만 blocking, dev는 주간 리포트로 분리해 게이트 우회 방지)
- gitleaks 이중 스캔(현재 트리 + PR 커밋 범위), CODEOWNERS(auth · BFF · workflows · env), 추적 env 파일 allowlist CI, 응답 보안 헤더

성과

- 배포 전 자동 검사 **0 → 17개**(6개 병렬 잡, 전부 통과해야 merge): lint · 포맷 · 타입 검사 · `exactOptionalPropertyTypes` 위반 상한 · 타입 레벨 테스트 · 단위 테스트 + 커버리지 하한 · 빌드 · e2e · 번역 파일 검사 3종(en.json과 코드 일치 · 키 규칙 · 설정 키) · Next 컨벤션 파일 위치 · 리다이렉트 origin · 시크릿 스캔 2종(현재 트리 · 푸시된 커밋) · 추적 env 파일 · 프로덕션 의존성 취약점
- lint **56 errors → 0**, 코딩 규칙 35개를 lint와 CI 검사로 강제

[2] 페이지 성능 최적화 (LCP · INP · 번들)

기술 스택 Lighthouse CI · Sentry · PerformanceEventTiming · @next/bundle-analyzer · Playwright
역할 성능 진단 · 개선 주도

문제

- 평가 상세 진입이 느리고 Layout Shift가 잦음

- 차트 라이브러리 recharts(gzip 압축 후 **116KB**)가 대시보드를 열지 않는 사용자에게도 초기 공유 번들로 배달
- 실사용자(필드) 지표와 랩 측정을 연결해 원인을 찾는 체계 부재

LCP, 필드 지표에서 원인까지

- Sentry의 Web Vitals에서 랜딩 `/` 의 LCP p75가 **5.3s**. 응답은 HTTP 200에 리다이렉트 0회
- 원인은 page 단계의 `redirect()` 가 RSC 페이로드로 직렬화돼 브라우저가 번들을 내려받고 하이드레이션한 뒤에야 이동하는 구조
- 리다이렉트를 Proxy(미들웨어) 계층으로 올려 마지막 방문 애플리케이션 쿠키 기준 **307**로 응답하게 바꿔 하이드레이션 구간을 LCP에서 제거
- 수정 전후 빌드를 같은 조건(Lighthouse 데스크톱 프로필 · 콜드 캐시 · 5회 median · CPU 1배와 3배)으로 비교해 랜딩 LCP **3.2 → 2.0s(–38%)**, 성능 점수 **81 → 90**. CPU 배수를 바꿔도 차이가 같아 병목이 연산이 아니라 하이드레이션을 기다리는 왕복이었음을 확인

LCP, 서버 프리페치

- 상세 페이지의 최대 콘텐츠 요소가 데이터 도착까지 **771ms** 렌더 지연
- Server Component에서 `prefetchQuery` 로 React Query 캐시를 채워 하이드레이션 직후 그리게 하고, 목록에서는 호버 시점에 prefetch
- 프리페치가 인증 헤더 없이 401을 받아 조용히 무효화되던 것도 트레이스로 잡아 수정
- 이미지 · 폰트 사이즈 예약과 스켈레톤으로 CLS 0

INP

- `PerformanceEventTiming` 을 페이지 안에서 수집하는 랩 하네스를 만들어 상호작용 지연을 입력 지연 / 처리 / 페인트로 분해(CPU 3~10배 스로틀, 계산 로직은 단위 테스트)
- 147행 Run 목록에서 행 하나를 펼칠 때 **320ms**(입력 지연 209ms)가 걸리는 것을 확인하고, 행마다 새로 만들어지던 콜백을 안정 참조로 바꾸고 펼침 상태 로직을 순수 함수로 분리

번들

- 처음엔 대시보드 컴포넌트를 지연 로딩(`dynamic()`)하면 recharts가 분리될 거라 봤는데, 다시 빌드해도 번들 파일 내용이 바이트 단위로 같아 효과가 없음을 확인
- 원인은 폴더의 index 파일이 대시보드를 함께 내보내고 있어서, 그 폴더의 다른 것을 쓰는 화면까지 recharts를 물고 들어오는 구조
- 지연 로딩 경계를 index 바깥이 아니라 위젯 안쪽에 두어 recharts를 초기 번들에서 분리
- 이 패턴을 팀 규칙으로 정하고, 어떤 모듈 · 패키지 · 파일이 얼마나 큰지 순위를 매기는 스크립트로 다음 개선 후보를 상시 볼 수 있게 함

PR 게이트

- Lighthouse CI(라우트별 3회 median, LCP · TBT · CLS, 인증 라우트는 CDP 쿠키 주입으로 측정) + main 대비 gzip 압축 크기 증감 diff · Top 15 패키지 랭킹을 PR 스티키 코멘트로 자동 게시
- 회귀 원인 모듈 추정 스크립트와 Claude Code 성능 리포트 스킬로 트리아지 시간 단축
- 예산 초과는 라우트별 기준 확정 전까지 경고 단계

성과

- 평가 상세 진입(User Timing 마크, 5회 median) 최악 케이스 **623 → 148ms(약 4배)**, 중앙값 **472 → 323ms**
- 대상 라우트 Lighthouse **99점 · CLS 0**(2026-07 실측, 같은 빌드 · URL 플래그 분기)
- 성능 회귀를 PR 단계에서 확인

[3] 레거시 아키텍처 전환 · 인증 · 타입 안전성

기술 스택 Next.js 14→16 · Route Handlers(BFF) · Middleware · Auth.js · Zustand · openapi-typescript · orval(Zod)
역할 아키텍처 설계 · 전환 주도

문제

- SSG 기반이라 서버에서 요청을 가로챌 수 없어 인증 검증 · 권한 라우팅이 구조적으로 불가능
- 전역 상태는 Recoil atom 429개로 흩어져 흐름 파악이 어렵고, API 타입을 수기로 관리해 백엔드 스펙 변경이 런타임에야 드러나는 상태

데이터 요청 경로

- 컴포넌트 → TanStack Query 훅 → `shared/api` 얇은 함수 → openapi-fetch(스펙 타입) → same-origin `/api/*` Route Handler(BFF) → 서버 세션에서 토큰 주입 → 백엔드. 이 한 경로만 허용하고 다른 경로는 lint로 차단
- 브라우저는 백엔드를 직접 호출하지 않고 토큰은 httpOnly 쿠키에만 두어 브라우저 JS가 읽을 수 없음
- 라우트마다 반복되던 요청 파싱 · 검증 · 프록시 코드를 공통 함수로 접어, 라우트 파일 270여 개가 대부분 40줄 안쪽

권한 · 세션

- Middleware로 **Company / Workspace / Application 3계층 권한(+슈퍼어드민)** 라우팅 가드

- 액세스 토큰 회전 + **24시간 절대 상한**, 리프레시 실패를 세션 종료(401)와 일시 오류(429 · 5xx)로 분기, 동시 401 폭주 시 리프레시를 하나로 묶어 늦은 요청의 강제 로그아웃 제거
- e2e 테스트가 잡은 버그 하나: 로그아웃해도 곧 다시 로그인 상태로 돌아옴. Auth.js가 세션을 읽을 때마다 쿠키를 새로 써 주는데, 로그아웃 직전에 출발한 요청(링크에 마우스를 올릴 때 미리 받아두는 RSC 프리페치 포함)이 뒤늦게 도착해 지운 쿠키를 되살리는 것이 원인
- 쿠키를 쓰는 세 곳(BFF 라우트 · Auth.js 핸들러 · 프리페치를 받는 Proxy)을 모두 "토큰이 실제로 바뀐 경우에만 쿠키를 쓴다"로 바꿔 해결

동시성 · 복원력

- 필터를 빠르게 바꾸면 먼저 보낸 요청의 응답이 나중에 도착해 새 화면을 덮는 문제를, 취소 신호(`AbortSignal`)를 fetch까지 전달해 이전 요청을 끊고 응답에 버전을 붙여 오래된 응답은 캐시에 쓰지 않게 해 해결
- 재시도는 일시 장애(타임아웃 · 429 · 5xx · 네트워크)에만 서버가 알려준 대기 시간을 지켜 간격을 늘리며 수행
- 두 번 실행되면 안 되는 액션(생성 · 삭제)은 연타 차단

상태 · 경계 · 패턴

- Recoil atom **429개 → Zustand 스토어 26개**. 기존 atom 이름을 selector로 감싸 화면 코드를 고치지 않고 도메인별로 옮겼고, Redux DevTools로 상태 변화를 추적할 수 있게 됨
- FSD **67개 슬라이스** 사이에 허용된 의존 방향을 ESLint 규칙(error)으로 강제해 feature 간 직접 import **39 → 0건**
- 서버 캐시 키를 손으로 쓰던 것을 도메인별 키 생성 함수(27개)로 통일해 조회 · 저장 후 무효화 · 낙관적 업데이트가 같은 키를 쓰게 함(키가 조금만 달라도 화면이 갱신되지 않던 문제 제거)
- API 경로 문자열도 슬라이스별로 모으고(20개), 백엔드 에러 응답은 하나의 에러 타입으로 변환해 화면 메시지까지 한 경로로 처리

타입 검사 강화

- TypeScript 기본 `strict` 위에 더 엄격한 옵션 4개를 추가로 켬(배열 인덱스로 꺼낸 값이 비어 있을 수 있음을 타입에 반영하는 `noUncheckedIndexedAccess` 등). 드러난 잠재 오류 **221건**을 전수 수정
- ESLint에도 타입 정보를 넘겨, 결과를 기다리지 않고 버린 Promise(109건 정리) · `any` 가 섞여 들어오는 값 사용 · `switch` 에서 빠진 경우를 오류로 처리
- 타입 검사를 우회하는 `as unknown as T` 표기는 lint로 금지

API 계약

- 백엔드에 OpenAPI 스펙 기반 타입 생성 도입을 제안. 요청 · 응답 타입을 사람이 손으로 옮겨 쓰던 것을 백엔드가 제공하는 OpenAPI 스펙에서 자동 생성하도록 바꿈(엔드포인트 300+ · 스키마 800+)
- 코드에 적은 API 경로가 스펙에 없으면 빌드가 실패하므로, 백엔드가 경로나 필드를 바꾸면 배포 전에 드러남

성과

- 정적 구조에서 불가능했던 인증 기반 라우팅 실현
- 신규 기능의 영향 범위를 슬라이스 단위로 격리
- 타입 불일치 런타임 에러를 CI 빌드 실패로 전환

[4] 핵심 기능 · 운영

기술 스택 SSE · TanStack Query · TanStack Table · shadcn/ui · ECharts/Recharts · next-intl · Sentry · Docker
역할 기능 설계 · 구현

Criteria AI 도구 4종

- Generator · Modifier · Validator · Augmenter로 평가 기준을 AI가 생성 · 수정 · 검증 · 증강
- 컬럼 단위와 전체 생성, 타깃 스텝 하이라이트, 템플릿 기반 Setup UI, 버전 히스토리, SSE 프레즌스 · 버전 충돌 기반 동시 편집
- 레포에서 가장 큰 기능 영역(349파일 중 232파일 작성)

인증 · 앱 셸

- 로그인 · 가입 · 비밀번호 재설정 · 이메일 인증 화면과 세션 모델 단독 구현
- LNB를 워크스페이스 역할 × 배포 타깃(SaaS / 온프레미스) × 테넌트 조합으로 게이팅, 슈퍼어드민 전역 사용자 검색 · 멤버 초대 · 권한 관리

실시간 협업

- 수 분~수십 분 걸리는 평가 진행 상황을 주기적으로 물어보는 폴링 대신 서버가 밀어주는 SSE로 전환
- 같은 문서를 여러 사람이 동시에 편집할 때 필요한 것들을 구현: 지금 누가 보고 있는지 표시, 내가 보낸 변경이 서버에서 되돌아와 화면이 두 번 바뀌는 것 방지(세션 ID로 자기 이벤트를 걸러냄), 다른 사람이 먼저 저장했으면 충돌을 알리고 최신 내용 반영, 탭을 닫을 때도 나갔다는 신호가 확실히 가게(`sendBeacon`)

- 워크스페이스를 바꿨는데 가끔 이전 값으로 되돌아가던 버그는 세 곳이 동시에 현재 값을 쓰던 경합이 원인이라, URL을 유일한 기준으로 삼는 전환 절차로 재설계

Red Teaming · Observability

- 공격셋 기반 자동 Red Teaming + human-in-the-loop 채팅으로 평가 → 취약점 탐지 → 재현을 한 흐름으로 연결
- Observability의 fail case 목록 · 상세 · 필터 · 커스텀 메트릭 설정 단독 구현

대용량 테이블 · 대시보드

- 무거운 그리드 라이브러리를 걷어내고 Headless(TanStack Table) + shadcn/ui로 가상화 · 2중 헤더 · rowspan/colspan 테이블 직접 구현
- Recharts · ECharts 차트와 드래그앤드랍 위젯 그리드로 사용자가 직접 배치하는 커스텀 대시보드 구현

운영 자동화

- i18n(next-intl) 단방향 파이프라인 구축. 코드가 `en` 원문을 키로 소유하고 번역자는 Google Sheet에서 `ja` · `ko` 만 편집하도록 규칙 설정
- AST 기반 미사용 키 정리와 CI 감사 3종, 컬럼 단위 소유권으로 약 1,300키에 번역 머지 충돌 0건
- Sentry를 server · edge · client 3런타임에 환경별 샘플링으로 통합하고 QueryCache `onError` 한 곳에서 중앙 계측, 3계층 노이즈 필터 · 소스맵 자동 업로드 · Slack 알림
- 5개 환경 배포 매트릭스 + 온프레미스 standalone Docker(금융권 포털 SSO 1회성 토큰 검증)로 SaaS · 온프레미스를 한 브랜치로 지원

주식회사 딥브레인AI · 프론트엔드 개발

2022.03 - 2025.03

AI 휴먼 영상 생성 SaaS **AI Studios** + 금융 · 공공 B2B SI (정규직 · 3년)

v2를 v3로 전면 리팩토링하며 Axios · prop drilling 구조를 Redux Toolkit + RTK Query로 옮기고 SSR · 디바운싱 · 코드 스플리팅을 적용해 **FCP / LCP / TTI 평균 30%+ 단축**. 캔버스 에디터 · 통계 대시보드 · 구독/결제 · 다국어 담당

1,000만 건 통계 대시보드

- 기간별 조회마다 raw 집계에 3초 이상 걸리던 것을 MongoDB 4축(날짜 · 기업 · 부서 · 유저) 사전 집계 + Kubernetes Cron 일배치로 단순 read 로 전환
- side-effect 플러그인이라 tree-shaking이 안 되던 Chart.js를 `dynamic()` 경계로 분리
- **FCP 1.2 → 0.4s(67%↓) · LCP 4.5 → 2.2s(51%↓) · 조회 3s+ → 1s 미만**

구독 · 해지 방어

- 레거시 · 신규 요금제가 병행되며 흩어진 권한 분기를 DB 필드 모델링 + 커스텀 훅으로 수렴, 6개 플랜 전환 규칙을 매트릭스로 명시
- 해지 클릭 시 유예 다운그레이드 → 할인 → 리워드 쿠폰 → 설문의 **4단계 퍼널**과 Stripe 결제 연동
- 출시 후 약 2개월간 신규 구독 **300명**

캔버스 웹 에디터(Fabric.js · FFmpeg)

- 매 드래그 프레임마다 전수 비교하던 정렬 스냅 가이드를 드래그 시작 1회 캐싱 + 축별 사전 필터링 + 가이드라인 DOM 분리로 바꿔 대형 캔버스 프레임 드랍 제거
- 비디오 트림(WASM PoC에서 서버 이관 판단), PPT → AI Video(Node.js ↔ Python 브리지), Undo/Redo 미들웨어, 생성형 AI 미디어 Job 폴링 반영, 청크 업로드
- Intersection Observer 무한 스크롤 + react-window 가상화, 에디터 진입 SSR 프리페치 + RTK Query 캐시 제어

통합 로그인 NPM 패키지 `@deepbrain/next-auth-account`

- NextAuth 기반 6종 OAuth(Google · MS · Apple · Facebook · **WeChat · Alipay RSA-SHA256 커스텀 서명**) + Redis 싱글턴 세션으로 서비스 간 SSO
- 설치만으로 적용되어 신규 서비스 인증 구축 비용 제거. 회원 어드민 · 중복 로그인 방지 · 탈퇴 30일 자동 삭제 배치 포함

그로스 · 글로벌

- GA4/GTM(Consent Mode) · Amplitude(Session Replay) 이벤트 트래킹과 A/B 테스트, Service Worker 웹푸시, AWS SES 드립 이메일(4 세그먼트 × 10+ 시나리오)
- next-i18next 10개국어 + Google Sheet ↔ JSON 자동 동기화, 위챗 로그인 · 알리페이 · Alibaba Cloud 배포 등 중국 리전 대응

금융 · 공공 SI

- 농협은행 태블릿 AI 실시간 통역(STT/TTS 10개국어 · ChatGPT 문맥 번역 · SDK 지연 로드로 **TTI 7.6 → 4.3s**)
- 농협 AI 허브 & 키오스크(자체 Vision SDK · PoseNet 정면 판별 · 폴라로이드 즉시 출력 · 온프레미스, 약 20억 규모)
- KB증권 드림아바타 PoC(ElevenLabs 음성 클로닝 · Webhook), 서울시교육청 AI 디지털 교과서(PC/모바일 이중 라우팅)

- 약 20억 규모 정부과제 학습 데이터 라벨링 · 검수 플랫폼(자동 라벨링 + 교차 검증 + 승인 파이프라인, **10만 장+**)

SKILL

구분	기술
Core	TypeScript · React · Next.js(App Router · SSR · BFF) · TanStack Query · Zustand · Redux/RTK Query
Quality · Perf	Vitest · Playwright · type-aware ESLint · Lighthouse CI · Sentry Web Vitals · bundle-analyzer · User Timing API
API · Auth	OpenAPI(openapi-typescript · openapi-fetch · orval) · Zod · SSE · NextAuth/Auth.js · Node.js · Redis · MongoDB
UI	shadcn/ui · Tailwind · Storybook · TanStack Table · ECharts/Recharts/Chart.js · Canvas/Fabric.js
Build · Ops	GitHub Actions · Docker · Kubernetes · AWS(S3 · SES) · pnpm · webpack/Turbopack · gitleaks · Dependabot
Growth · i18n	GA4/GTM · Amplitude · Stripe · next-intl/next-i18next
AI 개발 방법론	Claude Code · Codex · 작성자 ≠ 검증자 교차검증 하네스(`@datumo/agent-harness`) · MCP 기반 성능 분석 워크플로

EDUCATION · AWARDS · CERTIFICATES

기간	학교	전공	상태
2026.03 - 2027.08(예정)	고려대학교 융합데이터과학 대학원	융합데이터과학 (GPA 4.42/4.5)	재학 중
2016.03 - 2020.02	상명대학교	컴퓨터과학	졸업

교육 · 수상: 네이버 부스트캠프 AI Tech (2021) · 삼성 청년 SW 아카데미 SSAFY (2020) · 인공지능 스타트업 경진대회 특별상 (2021)
자격증: 정보처리기사 (2019) · SQLD (2020) · ADsP (2020)